An Experimental Comparison of Sudoku Solvers
Using Backtracking and Bitmasked Constraint Techniques

CSCI 4118 Final Project Report

Riham Otman — B00887629

March 2026

1. Introduction

Sudoku is a logic puzzle that can be modelled as a Constraint Satisfaction Problem (CSP), where each cell must satisfy row, column, and box constraints simultaneously. A basic backtracking solver can solve any valid puzzle. Still, its performance degrades sharply as difficulty increases, thus harder puzzles force the solver to explore an exponentially larger number of invalid states before finding a solution.

This project investigates how much performance can improve when naïve backtracking is replaced with an engineered alternative. The bitmasked constraint solver uses three algorithmic improvements: Minimum Remaining Values (MRV) heuristic ordering, forward checking, and forced singles propagation. It also uses an integer bitmask to represent row, column, and box constraints, reducing the number of candidate validity checks from O(n) to O(1). The main question is not whether Sudoku can be solved, but how the choices made when designing data structures and pruning strategies affect runtime, search effort, and scalability among different levels of difficulty.

Both solvers were implemented in Java and tested on 150 puzzles across three difficulty levels. Results confirmed the hypothesis: the bitmask solver achieved 17–24× faster runtimes and over 99.9% fewer backtracking steps among all difficulty levels.

2. Background and Literature Review

Norvig (2006/2021) showed that combining constraint propagation with recursive search solves Sudoku puzzles faster than plain backtracking alone. He modelled the puzzle as a CSP, where each square must meet the constraints of its row, column, and 3×3 box. Norvig specifically used recursive backtracking along with an elimination strategy called forward checking.

This project uses Norvig's method as a starting point, but it focuses more on low-level algorithm engineering. Norvig's Python implementation uses high-level list and dictionary structures to make things clearer, but this project uses bitmasks (integer bitsets) to set limits. Norvig himself said that this method was much faster for speeding up validity checks in his high-performance Java version (Norvig, 2006).

3. Algorithms and Techniques

3.1 Naïve Backtracking Solver

The naïve solver uses a normal recursive depth-first search. It goes from left to right and top to bottom on the board to find the first empty cell. Then it tries to assign values from 1 to 9 in order. It checks the validity of each candidate value by doing a linear scan of the cell's row, column, and 3×3 box. The solver goes back if there isn't a valid value. The simple solver doesn't have any lookahead or heuristic ordering.

3.2 Bitmasked Constraint Solver

The bitmasked solver employs integer bitsets to represent the possible values for each row, column and 3x3 box. In contrast, the naïve solver requires a linear scan that operates in O(n) time. A single bitwise AND operation across the three relevant masks, however, yields the complete candidate set for any cell in O(1) time.

The solver incorporates three key algorithmic enhancements:

- **Minimum Remaining Values (MRV) Heuristic**: selects the empty cell with the fewest candidates at each decision point, therefore maximizing the probability of a correct assignment.
- **Forward Checking**: immediately removes an assigned value from the candidate bitmasks of all neighbours or 'peers' (row, column, box), therefore detecting dead ends before recursion.
- **Forced Single Propagation**: repeatedly fills cells that have only one remaining candidate without consuming a backtracking step.

4. Experimental Setup

4.1 Dataset

I used 150 Sudoku puzzles from open-source Sudoku Exchange Puzzle Bank on GitHub categorized by difficulty: 50 Easy, 50 Medium, and 50 Hard (Sudoku Exchange Puzzle Bank, 2023). The first 50 entries from each difficulty category were selected and put into one csv file to ensure a consistent and reproducible benchmark.

4.2 Implementation

Both solvers were implemented in Java (JDK 23) and benchmarked on an Apple M4 processor with 24GB RAM. To account for JVM Just-In-Time (JIT) compilation effects, a warm-up phase of 100 non-recorded iterations was performed before the benchmark began.

4.3 Metrics

Each puzzle was solved 10 times per algorithm to account for system noise. The following metrics were recorded for each run:

- **Runtime (ms):** measured using System.nanoTime() and converted to milliseconds.
- **Backtracking Steps:** the number of times the recursive function returned due to a dead-end.
- **Total Assignments Attempted:** every value written to a cell, accumulating total search effort.

4.4 Statistical Methods

I used a paired t-test to compare the runtimes of both solvers on the same puzzles, which demonstrates that the performance differences are statistically significant. The mean runtime,

standard deviation, and speedup ratio (Naïve Runtime / Bitmask Runtime) were calculated for each level of difficulty.

5. Results

5.1 Summary Table

Difficulty	Naïve Mean (ms)	Naïve Std (ms)	Bitmask Mean (ms)	Bitmask Std (ms)	Speedup
Easy	0.374	1.019	0.020	0.033	17.1×
Medium	0.597	1.024	0.030	0.043	24.5×
Hard	0.661	0.988	0.045	0.047	20.4×

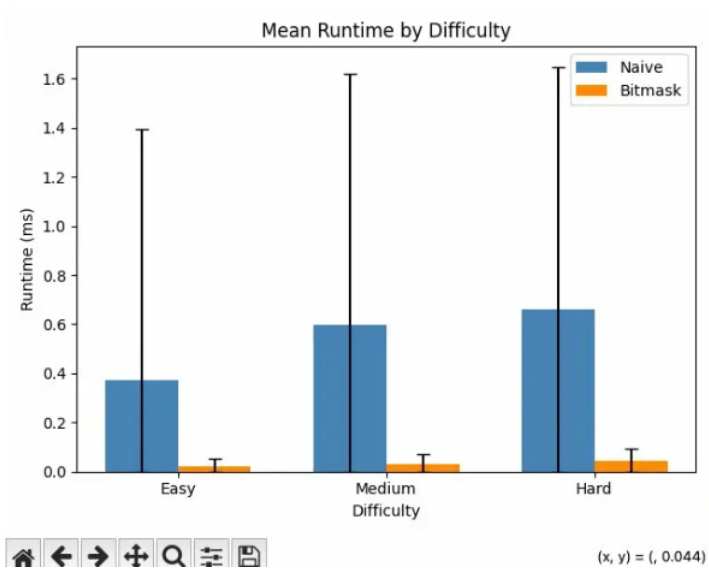

Figure 1: Mean Runtime by Difficulty (ms)

The runtime results clearly demonstrate that the bitmask solver consistently performs better, regardless of problem difficulty. The naïve solver took longer to solve problems, going from 0.374 milliseconds on Easy to 0.661 milliseconds on Hard. The bitmask solver, on the other hand, always stayed below 0.05 milliseconds during the whole experiment. Figure 1 shows that the naïve solver's standard deviation is higher than its mean at all levels of difficulty. Indicating that it runs slower and less consistently. The bitmask solver's average time went up from 0.020 milliseconds to 0.045 milliseconds across all three levels of difficulty. The naïve solver's average time went up from 0.374 milliseconds to 0.661 milliseconds.

5.2 Search Effort

Difficulty	Naïve Backtracks	Bitmask Backtracks	Naïve Assignments	Bitmask Assignments	Backtrack Reduction
Easy	8,487	5	8,539	105	99.94%
Medium	14,218	7	14,272	148	99.95%
Hard	15,829	11	15,882	214	99.93%

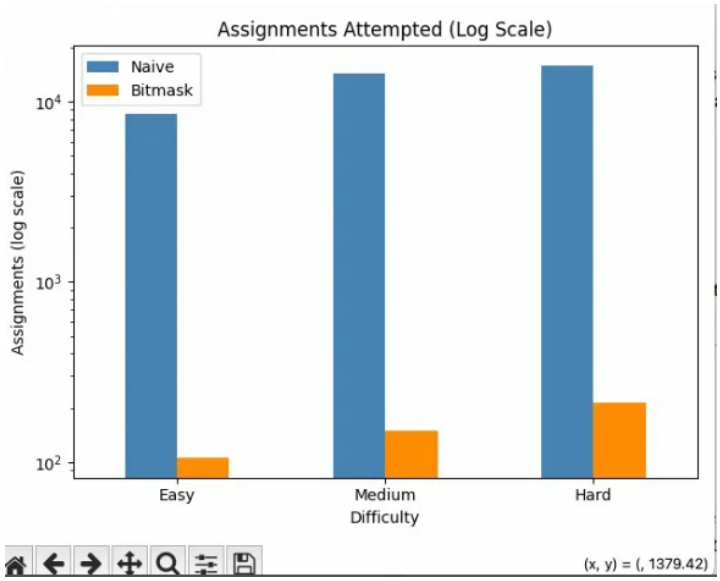

Figure 2: Assignments Attempted by Difficulty (Log Scale)

The bitmask solver needed only 5–11 backtracking steps for each level of difficulty, while the naïve solver needed 8,487–15,829. This means that the bitmask solver used 99.9% fewer steps (specifically, 99.93% in my experiment) than expected, which is much more than the at least 90% expected. The log-scale chart shows how big this difference is: the naïve solver's assignment counts are two to three orders of magnitude higher for all levels of difficulty. This occurs through MRV heuristic ordering and forward checking, which together prevent the solver from exploring subtrees that are guaranteed to fail before they are explored. MRV ensures the most constrained cell is always selected first, catching bad assignments early, while forward checking detects dead ends before recursion is invoked, meaning the solver almost never needs to backtrack because invalid branches are cut off before they are even explored. In practical terms, the bitmask solver avoided almost all useless searches that the naïve solver performed.

5.3 Statistical Significance

Scope	t-statistic	p-value	Significant?
Overall	6.2834	< 0.000001	Yes ($p < 0.05$)
Easy	2.5131	0.0153	Yes ($p < 0.05$)
Medium	3.9446	0.000254	Yes ($p < 0.05$)

Scope	t-statistic	p-value	Significant?
Hard	4.4297	0.000053	Yes ($p < 0.05$)

The paired t-test results confirm that observed performance differences are not due to random variation or outlier puzzles. The overall t-statistic of 6.2834 ($p < 0.000001$) is highly significant, and significance holds across every individual difficulty tier.

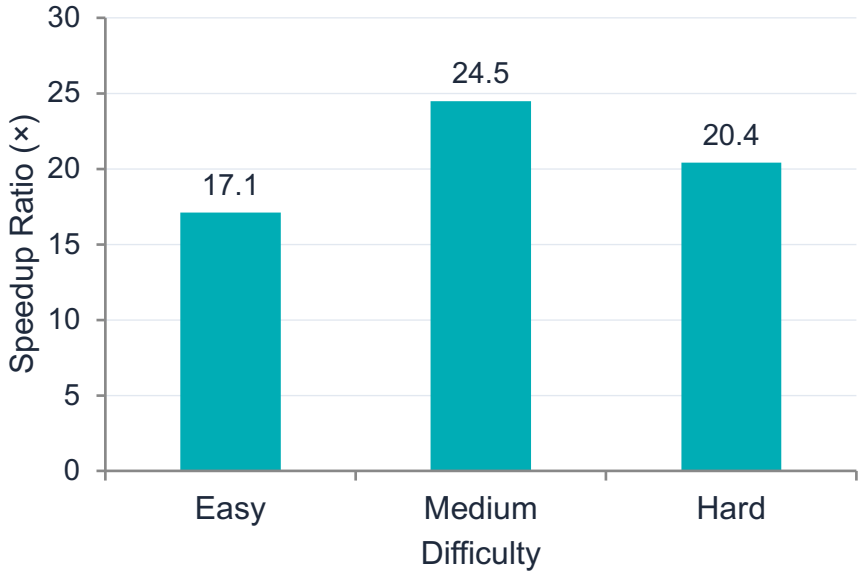

Figure 3: Average Speedup Ratio (Naïve / Bitmask)

6. Discussion

6.1 Runtime Performance

The speedup ratio didn't scale evenly across difficulty levels. It peaked at 24.5× on Medium, not Hard. On Easy puzzles, both solvers finish quickly, so the ratio mostly indicates that the bitmask's faster candidate lookups rather than any big difference in search behaviour. Medium is where the naïve solver starts struggling, as it hits more dead ends and explores many more invalid states, so the bitmask solver's pruning makes the biggest relative difference there. On Hard, the gap narrows slightly because forced singles propagation is doing a lot of the work, and the search space is more constrained to begin with, so even the naïve solver isn't wandering as freely. So the speedup isn't really about raw speed, therefore it's mostly about how much useless search each difficulty tier forces the naïve solver into.

6.2 Search Efficiency

Regarding search efficiency, backtracking steps were reduced by 99.93% across all tiers, exceeding the minimum expected results threshold. This is because MRV picks the most constrained cell first, so bad assignments get caught early before the search goes too deep. Forward checking makes this worse for the naïve solver as every assignment immediately

updates the candidate masks, so dead ends get detected before a recursive call is even made rather than after. Together, these two things mean the solver is mostly just heading straight to the solution, rather than wasting steps by exploring the space around it. It also explains the one unexpected result: backtrack reduction was actually lower on Hard than on Medium. Harder puzzles have fewer valid paths, so even the naïve solver doesn't wander as much as it does on moderately constrained boards.

6.3 Hypothesis Evaluation

My prediction in the hypothesis expected that the bitmask solver would show a significantly flatter runtime curve as difficulty increases, with at least a 90% reduction in backtracking on Hard puzzles. The hypothesis was proven and substantially outperformed by the results, as the actual outcome demonstrated a 99.93% reduction in backtracks on Hard puzzles—nearly 10 times the threshold.

6.4 Comparison with Prior Work

As noted in the background, Norvig's solver put readability ahead of speed and accepted that lower-level representations could produce better results. This project provides solid ground for that observation. Norvig showed that the algorithmic idea works, and this project shows what happens when it is made to run faster: a 17–24× improvement in runtime and a 99.9% reduction in backtracking steps compared to naïve backtracking. These gains are not small; they are statistically significant ($t = 6.2834$, $p < 0.000001$) and hold true for all three levels of difficulty. This provides concrete, measured evidence towards the direction Norvig pointed at but did not pursue due to the purpose of his work .

7. Conclusion

This study investigates 9x9 Sudoku solvers utilizing backtracking and bitmasked constraint techniques. It demonstrates that using a bitmask constraint representation, the MRV heuristic, and forward checking in algorithm engineering makes a significant difference in solving Sudoku puzzles, compared to naïve backtracking, in terms of runtime, backtracking steps, and total assignments. The bitmask solver ran 17–24 times faster and needed 99.9% fewer backtracking steps at all difficulty levels. All of these differences were found to be statistically significant by paired t-tests ($p < 0.05$). These results confirm the fundamental principle that shifting from a "working" algorithm to an "engineered" one, through the strategic application of data structures and constraint propagation, produces significant, quantifiable performance improvements. The methods shown here can be applied to other CSP problems where constraint-based pruning reduces the search space.

Previously, naïve backtracking made harder puzzles slow and unpredictable; with the bitmasked solver, even the most difficult 9×9 puzzles can now be solved in sub-millisecond time (under 0.05 ms) with near-zero wasted effort, making the approach suitable for real-time applications or massive batch-processing tasks.

While my experiment confirmed my hypothesis, some limitations arose. The naïve solver exhibited significant timing variability, with standard deviations exceeding the means in some

cases. This means that JVM background activity adds noise even after a warmup. To reduce this, each puzzle was run 100 times to warm up, then 10 times to record the results. However, there were still inconsistencies after the warm-up. This ties into another limitation: the dataset contains only 150 puzzles from a single source, which could limit the generalizability of this experiment. Additionally, another limitation that is not related to the experiment I conducted myself, but rather to the flexibility of applying the bitmask structure to larger grids, is the need to handle larger grids. Since the bitmask structure implemented only works with 9-bit integers, it can't be used with larger grids like 16×16 or 25×25 Hexadoku without major changes to the data structures.

Future research could enhance this study by analyzing larger datasets exceeding 500 puzzles sourced from various sources, which would help minimize the risk that difficulty ratings are inconsistent across puzzles. Employing more sophisticated propagation techniques such as AC-3, comparing performance across languages like Python and C++, or reimplementing the solvers in a logic programming language like Prolog could help more effectively isolate algorithmic improvements from language-specific optimizations. Adapting the bitmasking approach for N×N grids with flexible bit sets is a logical next step to determine whether the performance improvements hold as problem size grows.

8. Reproducibility

All code and data required to reproduce the results are provided. The following instructions assume a Unix-like environment (macOS or Linux).

Installation Requirements

I used Java JDK 23 or later (for compiling and running the solvers), Python 3.9 or later with the following packages: pandas, numpy, scipy, matplotlib (install via pip install pandas numpy scipy matplotlib).

Data

The 150 puzzles used (50 Easy, 50 Medium, 50 Hard) are included in the submission as puzzles.csv. They were taken from the Sudoku Exchange Puzzle Bank on GitHub (https://github.com/grantm/sudoku-exchange-puzzle-bank) by taking the first 50 entries from each difficulty category CSV file.

Running Instructions

- Compile and run SudokuBenchmarkRunner.java (javac SudokuBenchmarkRunner.java && java SudokuBenchmarkRunner). This reads puzzles.csv, runs 100 JVM warm-up iterations, then benchmarks each solver 10 times per puzzle, and writes results.csv.
- Copy results.csv into the Python project so that analysis.py can read it.
- Run analysis.py (python analysis.py). This reads results.csv and produces the summary statistics, paired t-test results, and all three figures used in this report.

9 References

Norvig, P. (2006). *Solving every Sudoku puzzle.* https://norvig.com/sudoku.html

Norvig, P. (2021). *Solving any Sudoku puzzle* [Updated notebook]. https://github.com/norvig/pytudes

Sudoku Exchange Puzzle Bank. (2023). *Sudoku exchange puzzle bank* [GitHub repository]. https://github.com/grantm/sudoku-exchange-puzzle-bank